Assignment: Build a Query-Expansion + Reranking RAG Pipeline from Scratch

Assignment Brief and Build Walkthrough | Domain: Product Management (Tiffinly Foods, a fictional company) | Reference repo: github.com/SetuAI/RAG_with_QueryExpansion_Reranker

Level	Time	Stack	You hand in
Beginner to Intermediate	6 - 8 hours	Python, LangChain, FAISS, OpenAI, sentence-transformers	5 Python files + notes.md + completed questionnaire + screenshots

The goal in one line. You will write, file by file, a question-answering system over three product-management PDFs. It rewrites each question in several ways, searches the documents with every version, removes duplicate results, re-scores what it found with a second model, and asks GPT-4o to answer using only the best chunks. This brief tells you what goes into each file and how to check it works. It does not give you the finished code.

1. The scenario

You have joined Tiffinly, a home-cooked meal subscription app, as its first AI engineer. Product managers keep asking the same kinds of questions: "What is the skip cut-off?", "Did the pause feature reduce churn?", "Why was Dietician Chat pushed back?". The answers are spread across three documents: a Product Requirements Document (PRD), the FY27 Roadmap and OKRs, and a User Research and Experiment Readout. Your job is to build an assistant that answers these questions correctly and shows which page each answer came from.

There is one real difficulty. The documents are written in product-management vocabulary ("monthly subscriber churn", "DAU", "North Star Metric", "guardrail"), while people ask in everyday words ("how many people quit each month?", "how many people open the app daily?"). A plain vector search often misses the right chunk when the words do not match. Query expansion fixes the missed chunks; reranking puts the right chunk first.

2. What happens when a user asks a question

#	Step	File	What goes in	What comes out
0	Ingest (run once)	ingest.py	3 PDFs in data/	faiss_index/ folder (index.faiss + index.pkl)
1	Expand	query_expansion.py	User question	Original question + 5 rewritten versions = 6 queries
2	Search	retriever.py	Each of the 6 queries	6 chunks per query = 36 chunks
3	Merge and dedupe	query_expansion.py	36 chunks	Only unique chunks (typically 8 - 15)
4	Rerank	reranker.py	Original question + unique chunks	Top 5 chunks, best first, each with a score
5	Answer	answer_chain.py	Top 5 chunks + original question	Answer with source file and page numbers

If any step returns the wrong chunks, every step after it is working with wrong material. That is why you build and check one file at a time, in the order of Section 5 onwards, and never move on until the checkpoint for that file passes.

3. The documents you are given

File (put in data/)	Pages	What it contains
Tiffinly_PRD_Smart_Pause_v2.pdf	2	Problem, goals G1-G3, guardrails, personas, requirements FR-01 to FR-10 with priorities, non-functional requirements, dependencies, rollout plan, risks, open questions OQ-1 and OQ-2, analytics events.
Tiffinly_Product_Roadmap_OKRs_FY27.pdf	2	Vision, North Star Metric (Weekly Meals Delivered), baseline metrics (DAU, MAU, churn, NPS, CAC, LTV, ARPU), OKRs O1-O3, RICE scores, quarterly roadmap, squads, decision log D-07 to D-09, quarterly targets.
Tiffinly_User_Research_Experiment_Readout_Q1FY27.pdf	2	Research methods, insights I1-I5, jobs to be done, NPS and complaint themes, usability test, A/B test EXP-0412 (Smart Pause), pricing test EXP-0419 (Family Plan), Spice Meter results, user quotes.

Read the three PDFs yourself before writing any code (about 15 minutes). You cannot judge whether a retrieved chunk is right if you do not know where the answer lives. Several facts deliberately appear in more than one document (for example NPS 41 in the roadmap and NPS 44 in the readout), and some answers need two documents together.

4. Setup

4.1 Folder structure

```
tiffinly-rag/
```

```
├── data/
│   ├── Tiffinly_PRD_Smart_Pause_v2.pdf
│   ├── Tiffinly_Product_Roadmap_OKRs_FY27.pdf
│   └── Tiffinly_User_Research_Experiment_Readout_Q1FY27.pdf
├── .env                  # OPENAI_API_KEY=sk-...   (never commit this file)
├── .gitignore            # .env, faiss_index/, __pycache__/, .venv/
├── requirements.txt
├── ingest.py             # Step 1  - PDFs -> chunks -> embeddings -> faiss_index/
├── retriever.py          # Step 2  - load faiss_index/ and search it
├── answer_chain.py       # Step 3  - chunks + question -> GPT-4o answer
├── reranker.py           # Step 4  - re-score chunks with a cross-encoder
├── query_expansion.py    # Step 5  - full pipeline (expand -> search -> merge -> rerank -> answer)
└── notes.md              # your observations, used to answer the questionnaire
```

4.2 Environment and dependencies

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
# requirements.txt contents:
# langchain  langchain-openai  langchain-community  langchain-text-splitters
# pypdf  python-dotenv  faiss-cpu  rich  sentence-transformers
```

Create **.env** with one line, `OPENAI_API_KEY=sk-...`. Every script calls `load_dotenv(override=True)` so the key is read from this file; the key never appears in code. Expected cost for the whole assignment, including all tuning runs: well under USD 1.

5. Step 1 - ingest.py: PDFs in, searchable index out

Job of this file: read every PDF page, cut the text into overlapping chunks, convert each chunk into an embedding (a list of 3,072 numbers for text-embedding-3-large), and save all of it to disk as a FAISS index. No other file ever opens a PDF; they only read faiss_index/. **Run it once**, and again only when you change the PDFs or the chunk settings.

What goes in, top to bottom: (1) the KMP_DUPLICATE_LIB_OK line, before any import of faiss; (2) load_dotenv; (3) imports; (4) one CONFIGURATION block holding every tunable value; (5) four functions, one per stage; (6) main() calling them in order. Fill in the TODOs:

```
import os
os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")   # must run BEFORE faiss is imported (macOS OpenMP clash)

from dotenv import load_dotenv
load_dotenv(override=True)                              # reads OPENAI_API_KEY from .env

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from rich.console import Console

console = Console()

# ── CONFIGURATION: every tunable value lives here and nowhere else ──
DOCUMENTS_DIR = "./data"
FAISS_INDEX_PATH = "./faiss_index"
CHUNK_SIZE = 700                  # characters per chunk
CHUNK_OVERLAP = 250               # characters shared by neighbouring chunks
EMBEDDING_MODEL = "text-embedding-3-large"   # must match retriever.py exactly

def load_documents(documents_dir: str) -> list:
    """Return one LangChain Document per PDF page, for every .pdf in documents_dir."""
    # TODO 1: build a list of paths for files whose name ends in ".pdf" (case-insensitive)
    # TODO 2: if the list is empty, raise FileNotFoundError with a message that names documents_dir
    # TODO 3: loop over sorted(paths); for each, PyPDFLoader(path).load() and extend one big list
    # TODO 4: print how many PDFs and how many pages were loaded, then return the list
    ...

def split_documents(documents: list) -> list:
    """Cut page-level Documents into overlapping chunks. Metadata (source, page) is kept."""
    # TODO 5: create RecursiveCharacterTextSplitter(chunk_size=..., chunk_overlap=...,
    #         separators=["\n\n", "\n", ". ", " ", ""])
    # TODO 6: chunks = splitter.split_documents(documents); print len(chunks)
    # TODO 7: print the first 400 characters of chunks[0] so you can inspect a chunk
    ...

def embed_and_store(chunks: list) -> FAISS:
    """Embed every chunk and build an in-memory FAISS index."""
    # TODO 8: embedding_model = OpenAIEmbeddings(model=EMBEDDING_MODEL)
    # TODO 9: vector_store = FAISS.from_documents(documents=chunks, embedding=embedding_model)
    # TODO 10: print vector_store.index.ntotal (number of vectors) and return vector_store
    ...

def save_vector_store(vector_store: FAISS, path: str) -> None:
```

```
    """Write index.faiss (vectors) and index.pkl (chunk text + metadata) into path."""
    # TODO 11: vector_store.save_local(path)
    ...

def main():
    documents = load_documents(DOCUMENTS_DIR)
    chunks = split_documents(documents)
    vector_store = embed_and_store(chunks)
    save_vector_store(vector_store, FAISS_INDEX_PATH)

if __name__ == "__main__":
    main()
```

Checkpoint 1. `python ingest.py` prints: 3 PDF files found, 6 pages loaded, about 40 - 45 chunks created, a sample chunk, and the same number of vectors as chunks. A faiss_index/ folder appears containing index.faiss and index.pkl. Run it a second time: the counts must be identical. If the chunk count changes between runs, something is not deterministic - find out what before moving on.

Look closely at the sample chunk. It starts with the page footer line and "Page 1", and table cells arrive one per line. This is how PyPDFLoader turns a laid-out PDF into plain text. You will be asked about it in the questionnaire (B3).

6. Step 2 - retriever.py: load the index and search it

Job of this file: load faiss_index/ from disk and expose one function, `get_retriever()`, that every other file imports. Keeping the search settings (embedding model, TOP_K) in this single place guarantees that reranker.py and query_expansion.py search in exactly the same way. Keep the same top-of-file lines as ingest.py (KMP line, load_dotenv, imports), then:

```
FAISS_INDEX_PATH = "./faiss_index"
EMBEDDING_MODEL = "text-embedding-3-large"   # same model as ingest.py, or the scores mean nothing
TOP_K = 6                                    # chunks returned per search

def load_vector_store(index_path: str) -> FAISS:
    # TODO 1: if index_path does not exist, raise FileNotFoundError("... run ingest.py first")
    # TODO 2: FAISS.load_local(folder_path=index_path, embeddings=OpenAIEmbeddings(model=EMBEDDING_MODEL),
    #                          allow_dangerous_deserialization=True)
    #         (True is safe ONLY because index.pkl is a file you created yourself)
    ...

def build_retriever(vector_store: FAISS):
    # TODO 3: return vector_store.as_retriever(search_type="similarity", search_kwargs={"k": TOP_K})
    ...

def get_retriever():
    """The ONE function other files import, so every file searches with identical settings."""
    return build_retriever(load_vector_store(FAISS_INDEX_PATH))

def display_retrieved_chunks(query: str, chunks: list) -> None:
    # TODO 4: for each chunk print: rank, os.path.basename(metadata["source"]),
    #         page number (metadata["page"] + 1, because PyPDFLoader counts from 0), first 300 chars
    ...

def main():
    retriever = get_retriever()
    query = input("Enter your query: ")
    display_retrieved_chunks(query, retriever.invoke(query))
```

Checkpoint 2. `python retriever.py`, then ask "*What is the skip cut-off time?*". Six chunks are printed. At least one is from Tiffinly_PRD_Smart_Pause_v2.pdf and contains FR-02 (9:00 PM). Now ask "*What is the last time I can cancel tomorrow's lunch?*" and record whether FR-02 still appears. This pair is your first look at the vocabulary gap.

Two classic bugs here. (a) Using a different EMBEDDING_MODEL than ingest.py: the code runs, but the results are nonsense, because vectors from different models cannot be compared. (b) Printing `metadata['page']` without + 1: every page number you report will be one too low.

7. Step 3 - answer_chain.py: turn chunks into an answer

Job of this file: hold the answer prompt and the GPT-4o call in one place, so reranker.py and query_expansion.py share the same answering logic. It has **no main()** and is never run directly. The prompt is written for the Tiffinly product domain: it restricts the model to the context, asks for exact IDs and numbers, and requires page citations.

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

ANSWER_MODEL = "gpt-4o"
ANSWER_TEMPERATURE = 0          # 0 = most repeatable answer; we want facts, not creativity
```

```
def format_context(chunks: list) -> str:
    """Join chunks into one text block, each labelled [SOURCE: file | PAGE: n]."""
    # TODO 1: if chunks is empty, return "No context retrieved."
    # TODO 2: for each chunk build f"[SOURCE: {file} | PAGE: {page}]\n{chunk.page_content}\n"
    # TODO 3: return "\n\n".join(parts)
    ...

SYSTEM_PROMPT = """You are a product analyst assistant for Tiffinly Foods.
You answer questions about Tiffinly's product requirements, roadmap, OKRs, user research and experiments.
Answer ONLY from the context below. Quote numbers, dates and IDs (FR-03, EXP-0412, KR1.1) exactly.
End every answer with the source file name(s) and page number(s) you used.
If the answer is not in the context, reply exactly:
"I could not find that information in the Tiffinly product documents."

CONTEXT:
{context}"""

def build_answer_chain(model: str = ANSWER_MODEL, temperature: float = ANSWER_TEMPERATURE):
    # TODO 4: prompt = ChatPromptTemplate.from_messages([("system", SYSTEM_PROMPT), ("human", "{question}")])
    # TODO 5: return prompt | ChatOpenAI(model=model, temperature=temperature) | StrOutputParser()
    ...

# No main(): this file is imported by reranker.py and query_expansion.py, never run directly.
```

The exact refusal sentence matters: in the questionnaire you will ask questions the documents cannot answer and check that the model replies with this sentence instead of guessing.

8. Step 4 - reranker.py: put the best chunk first

Job of this file: take the chunks a search returned and give each one a fresh relevance score. The FAISS search compared two vectors that were calculated separately; the question and the chunk were never read together. A cross-encoder takes the question and one chunk as a single input and outputs one score, so it judges relevance more accurately. It is slower, so it is only run on the few chunks already retrieved, never on the whole index.

```
RERANKER_MODEL = "cross-encoder/ms-marco-MiniLM-L-6-v2"   # ~80 MB, downloaded once on first use
TOP_N = 5                                                 # chunks kept after reranking
_reranker_model = None                                    # cache: load the model only once

def get_reranker_model():
    global _reranker_model
    # TODO 1: if _reranker_model is None: import CrossEncoder from sentence_transformers,
    #         set _reranker_model = CrossEncoder(RERANKER_MODEL)
    return _reranker_model

def rerank(query: str, chunks: list, top_n: int = TOP_N, show_comparison: bool = True) -> list:
    # TODO 2: if not chunks: return []          (do not load the model for nothing)
    # TODO 3: pairs = [[query, c.page_content] for c in chunks]
    # TODO 4: scores = get_reranker_model().predict(pairs)
    # TODO 5: scored = [{"chunk": c, "score": float(s), "original_rank": i + 1}
    #                   for i, (c, s) in enumerate(zip(chunks, scores))]
    # TODO 6: sort scored by "score", highest first
    # TODO 7: if show_comparison: print a BEFORE list (original order) and an AFTER list showing
    #         new rank, score, "was #x", movement (original_rank - new_rank) and KEPT / dropped
    # TODO 8: return [item["chunk"] for item in scored[:top_n]]
    ...

def main():
    from retriever import get_retriever
    from answer_chain import format_context, build_answer_chain
    # TODO 9: retrieve for one typed query -> rerank -> format_context -> answer_chain.invoke(
    #         {"context": ..., "question": ...}) -> print the answer
    ...
```

Checkpoint 3. `python reranker.py` with "*How many people quit their subscription every month?*". The first run downloads the model (about 80 MB). You see a BEFORE list (6 chunks, search order) and an AFTER list with scores, movements such as ^3 or v2, and 5 KEPT / 1 dropped. The answer should say 9.4% and cite the roadmap (and may add the 7.3% test result from the readout). Screenshot the BEFORE / AFTER block.

Always rerank against the user's original question, never against an expanded phrasing. The rewritten versions exist to find more candidates; the original question is what the user actually wants answered.

9. Step 5 - query_expansion.py: the full pipeline

Job of this file: connect everything. It imports `get_retriever`, `rerank`, `format_context` and `build_answer_chain` from your other files and runs the six steps of Section 2 for every question typed. The expansion prompt is tuned for product-management vocabulary.

```
EXPANSION_MODEL = "gpt-4o-mini"   # cheap, fast model: rephrasing does not need the big model
EXPANSION_TEMPERATURE = 0.4       # a little randomness so the phrasings differ from each other
NUM_EXPANSIONS = 5
RERANK_TOP_N = 5
```

```
EXPANSION_PROMPT = """You rewrite questions about a food-subscription company's product documents
(PRDs, roadmaps, OKRs, user research, A/B test readouts) to improve document search.
Generate {num_expansions} alternative phrasings of the question below that mean the same thing.
Use product-management vocabulary a PM would write (for example churn, DAU, NPS, CAC, OKR, KR,
North Star Metric, guardrail, P0, RICE) as well as plain everyday wording.
Question: {question}
Output ONLY the {num_expansions} phrasings, one per line. No numbering, no explanation."""

def build_query_expander():
    # TODO 1: ChatPromptTemplate.from_template(EXPANSION_PROMPT) | ChatOpenAI(...) | StrOutputParser()
    ...

def expand_query(original_query: str, num_expansions: int = NUM_EXPANSIONS) -> list:
    # TODO 2: raw = build_query_expander().invoke({"question": original_query,
    #                                              "num_expansions": num_expansions})
    # TODO 3: split raw into lines, strip each, drop empty lines
    # TODO 4: return [original_query] + phrasings        (the original question is ALWAYS searched)
    ...

def retrieve_with_expansion(original_query: str, retriever) -> list:
    all_queries = expand_query(original_query)
    seen_content = set()     # text of every chunk already kept
    merged_chunks = []       # unique chunks, in the order they were first found
    for i, query in enumerate(all_queries):
        chunks = retriever.invoke(query)
        # TODO 5: for each chunk: if chunk.page_content not in seen_content -> add to the set,
        #         append to merged_chunks, label it NEW; otherwise label it duplicate.
        # TODO 6: print "Query i: <query>" and, per chunk, [NEW]/[duplicate] + source + page + preview
    # TODO 7: print f"Merged result: {len(merged_chunks)} unique chunk(s) across {len(all_queries)} queries"
    return merged_chunks

def main():
    retriever = get_retriever()
    answer_chain = build_answer_chain()
    while True:
        query = input("\nEnter the query (or exit): ").strip()
        if query.lower() in ("exit", "quit", "q"):
            break
        if not query:
            continue
        merged = retrieve_with_expansion(query, retriever)
        top_chunks = rerank(query, merged, top_n=RERANK_TOP_N)   # rerank against the ORIGINAL question
        # TODO 8: answer = answer_chain.invoke({"context": format_context(top_chunks), "question": query})
        # TODO 9: print the answer
```

Checkpoint 4. Ask "*What's the one number the whole company is trying to grow?*". The screen shows 6 queries (one marked original), one search per query with NEW / duplicate tags, a line like "Merged result: 11 unique chunk(s) across 6 queries", the BEFORE / AFTER rerank block, and an answer naming Weekly Meals Delivered (1.42 lakh baseline, 2.10 lakh target) with the roadmap as source. Your exact chunk counts will differ, because expansion uses temperature 0.4.

Two settings that are easy to confuse. NUM_EXPANSIONS = 5 decides how many *searches* run (5 + the original = 6). TOP_K = 6 decides how many chunks *each* search returns. They are independent. Change one and check that only the matching number on screen changes.

10. Step 6 - test, tune and record

Use this question bank for every experiment. Each pair asks the same thing twice: once in the documents' own words, once in everyday words. Keep a table in notes.md with, for each question and each pipeline (retriever only, reranker, full pipeline): was the right chunk retrieved, its rank, and whether the answer was correct.

ID	Document wording	Everyday wording	Correct answer (where to verify)
Q1	What is the DAU?	How many people open the app every day?	61,000 (Roadmap, baseline metrics)
Q2	What is the monthly subscriber churn?	How many subscribers quit each month?	9.4% baseline (Roadmap; PRD section 2)
Q3	What is the maximum pause duration in FR-03?	For how long can I stop my meals?	2 to 21 days per pause (PRD, FR-03)
Q4	What is the North Star Metric?	What's the one number the company wants to grow?	Weekly Meals Delivered: 1.42 lakh to 2.10 lakh/week (Roadmap)
Q5	What was the result of EXP-0412?	Did the new pause feature stop people leaving?	Churn 9.6% to 7.3%, -24% relative, p = 0.003; shipped 14 July 2026 (Readout)
Q6	Why was Dietician Chat deprioritised (D-07)?	Why was the nutrition chat pushed back?	40% confidence + regulatory review of nutrition advice (Roadmap, decision log)

ID	Document wording	Everyday wording	Correct answer (where to verify)
Q7	What is the skip cut-off for Hyderabad?	When is the last time someone in Hyderabad can cancel tomorrow's lunch?	8:00 PM previous day; other cities 9:00 PM (Readout, answers PRD OQ-2)
Q8	Did Smart Pause meet KR1.1?	Did the pause feature hit its churn goal?	Not yet: 7.3% vs 7.0% target; save flow FR-09 expected to close the gap (needs 2 documents)
Q9	-	What is Tiffinly's share price?	Not in the documents: must return the refusal sentence

Tuning runs (one change at a time, always re-run ingest.py after changing chunk settings): CHUNK_SIZE / CHUNK_OVERLAP of 300 / 50, 700 / 250 (default) and 1200 / 200; TOP_K of 3 and 10; NUM_EXPANSIONS of 0, 3 and 5; RERANK_TOP_N of 2 and 8. The questionnaire (Section F) lists exactly what to record for each.

11. Troubleshooting

Symptom	Cause	Fix
OMP Error #15 ... libomp already initialized	faiss and torch each load OpenMP (mostly macOS)	KMP_DUPLICATE_LIB_OK line must be the first lines of the file, before importing faiss
FileNotFoundError: FAISS index not found	ingest.py not run, or run from another folder	Run python ingest.py from the project root first
ValueError ... allow_dangerous_deserialization	FAISS refuses to unpickle index.pkl by default	Pass allow_dangerous_deserialization=True (only for an index you built)
AuthenticationError / 401	Key missing or wrong in .env	Check .env sits in the project root; no quotes or spaces
Retrieval looks random	Different embedding models in ingest.py and retriever.py	Use the same EMBEDDING_MODEL; delete faiss_index/ and re-ingest
Merged result always equals 6	Expansion returned nothing, or lines were not split	Print the raw expansion output; check the split on newlines
Page numbers are one too low	PyPDFLoader pages start at 0	Display metadata['page'] + 1

12. What to submit and how it is graded

Item	Marks	What earns full marks
ingest.py and retriever.py	15	Checkpoints 1 and 2 pass; all settings in the config block; clear errors for missing PDFs/index
answer_chain.py	10	Answers cite file and page; Q9 returns the exact refusal sentence
reranker.py	15	Correct scores and ordering; BEFORE/AFTER shows movement and KEPT/dropped; model loaded once
query_expansion.py	15	6 queries, correct dedupe counts, reranks against the original question, loop with exit
Questionnaire answers	35	Every answer backed by an observation from your own runs (numbers, ranks, screenshots)
Code quality	10	Docstrings, no hard-coded values outside config blocks, no secrets in code, runs from a clean clone

Submit: a zip or GitHub link with the five .py files, requirements.txt, notes.md and a screenshots/ folder (at least: ingest output, one BEFORE/AFTER rerank block, one full-pipeline run). Do **not** submit .env or faiss_index/. Answer the questionnaire in a separate document, using the question IDs (A1, B2 ...).

13. Stretch goals (optional, no extra marks)

- Write `evaluate.py`: run Q1-Q9 through all three pipelines and print a hit-rate table (was the expected page in the final top 5?).
- Strip the repeated footer line from each page in load_documents() before chunking, then compare retrieval on Q1 and Q2.
- Replace the content-based dedupe key with (source, page, start index) by passing add_start_index=True to the splitter. What changes?
- Time each step with time.perf_counter() and report which step dominates the total latency.